

Informe breve — Unidad 4: FNBC y desnormalización controlada

Motor: PostgreSQL 16+ — **Repo:** BDD-2PRO1-GRUPO-F — **Scripts:** [tp_fnbc_control_lote.sql](#), [tp_desnormalizacion_top_categorias.sql](#)

Parte 1 — FNBC control_lote_almacen

a) Dependencias funcionales

- DF1: {LoteID, DepositoID} -> ResponsableControlID. PK declarada.
- DF2: ResponsableControlID -> DepositoID. Cada responsable pertenece a un único depósito.
- No se postulan LoteID -> DepositoID ni LoteID -> ResponsableControlID (un lote se controla desde varios depósitos).

b) Clausuras y claves candidatas

$U = \{L, D, R\}$. $\{L, D\}^+ = \{L, D, R\}$ por DF1, mínima => candidata. $\{R\}^+ = \{R, D\}$, no cubre. $\{L, R\}^+ = \{L, R, D\}$ por DF2, mínima => candidata. Claves: $\{\{LoteID, DepositoID\}, \{LoteID, ResponsableControlID\}\}$. Primos: los 3. No primos: ninguno.

c) Violación FNBC

Definición: R en FNBC si toda DF no trivial $X \rightarrow Y$ tiene X superclave. $\{L, D\} \rightarrow R$ sí (superclave), $R \rightarrow D$ no ($\{R\}^+ = \{R, D\} \neq U$). Esquema NO en FNBC. Violatoria: ResponsableControlID -> DepositoID.

d) Anomalías (instancia 501/502/503)

- Inserción: no se puede registrar 803 en depósito 32 sin lote ficticio (lote_id parte de PK, NOT NULL).
- Borrado: borrar (503,31,802) borra la única evidencia de que 802 pertenece a 31.
- Actualización: mover 801 de depósito 30 a 33 exige 2 filas (501,502); si falla una, inconsistencia.

e-f) Descomposición y sin pérdida

Heath sobre $R \rightarrow D$: R1 responsable_deposito(R PK, D), R2 control_lote(L, R PK). Vista v_control_lote_almacen = JOIN sobre responsable_control_id. Común es PK en R1 => superclave en R1 => join sin pérdida, verificado con EXCEPT = 0 filas tras migrar con SELECT DISTINCT.

Parte 2 — Top 5 categorías del día

a) Medición original

EXPLAIN (ANALYZE, BUFFERS) sobre instancia local con venta del día (pedido 2, rows=1). Nodo dominante: Hash Join + Seq Scan.

```
-----
QUERY PLAN
-----
Limit (cost=9.51..9.51 rows=1 width=250) (actual time=0.122..0.124 rows=1.00 loops=1)
 Buffers: shared hit=4 read=1
-> Sort (cost=9.51..9.51 rows=1 width=250) (actual time=0.122..0.123 rows=1.00 loops=1)
   Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC
   Sort Method: quicksort Memory: 25kB
   Buffers: shared hit=4 read=1
-> GroupAggregate (cost=9.47..9.50 rows=1 width=250) (actual time=0.117..0.118 rows=1.00 loops=1)
   Group Key: c.nombre
   Buffers: shared hit=4 read=1
-> Sort (cost=9.47..9.47 rows=1 width=238) (actual time=0.111..0.112 rows=1.00 loops=1)
   Sort Key: c.nombre
   Sort Method: quicksort Memory: 25kB
   Buffers: shared hit=4 read=1
-> Nested Loop (cost=6.09..9.46 rows=1 width=238) (actual time=0.104..0.106 rows=1.00 loops=1)
   Buffers: shared hit=4 read=1
-> Hash Join (cost=5.94..7.28 rows=1 width=28) (actual time=0.060..0.061 rows=1.00 loops=1)
   Hash Cond: (pr.id = dp.producto_id)
   Buffers: shared hit=3
-> Seq Scan on producto pr (cost=0.00..1.24 rows=24 width=16) (actual time=0.016..0.016 rows=2.00 loops=1)
   Buffers: shared hit=1
-> Hash (cost=5.93..5.93 rows=1 width=28) (actual time=0.036..0.036 rows=1.00 loops=1)
   Buckets: 1024 Batches: 1 Memory Usage: 9kB
   Buffers: shared hit=2
-> Hash Join (cost=3.52..5.93 rows=1 width=28) (actual time=0.030..0.031 rows=1.00 loops=1)
   Hash Cond: (dp.pedido_id = ped.id)
   Buffers: shared hit=2
-> Seq Scan on detalle_pedido dp (cost=0.00..2.25 rows=62 width=36) (actual time=0.013..0.014 rows=2.00 loops=1)
   Buffers: shared hit=1
   Filter: (NOT eliminado)
   Buffers: shared hit=1
-> Hash (cost=3.50..3.50 rows=1 width=8) (actual time=0.009..0.009 rows=1.00 loops=1)
   Buckets: 1024 Batches: 1 Memory Usage: 9kB
   Buffers: shared hit=1
-> Seq Scan on pedido ped (cost=0.00..3.50 rows=1 width=8) (actual time=0.006..0.006 rows=1.00 loops=1)
   Filter: ((NOT eliminado) AND ((fecha)::date = CURRENT_DATE))
   Rows Removed by Filter: 1
   Buffers: shared hit=1
-> Index Scan using categoria_pkey on categoria c (cost=0.15..2.17 rows=1 width=226) (actual time=0.043..0.043 rows=1.00 loops=1)
   Index Cond: (id = pr.categoria_id)
   Index Searches: 1
   Buffers: shared hit=1 read=1
-- More --
```

b) Patrón elegido: vista materializada

La evidencia de 4 JOINS + GROUP BY ejecutados por minuto motiva pre-agregar por (dia, categoria). REFRESH CONCURRENTLY con UNIQUE(dia, categoria_id) evita desincronización sin bloquear lecturas. Reversible con DROP, la fuente queda intacta. Se descarta columna con trigger por overhead en cada INSERT.

c-d) Implementación y mejora

Estructura en `tp_desnormalizacion_top_categorias.sql`: mv_ventas_dia_categoria + UNIQUE + consulta WHERE dia=CURRENT_DATE LIMIT 5 (3 filas vs 300k).

	Antes (4 JOINS)	Después (MV)
Tiempo	0.124 ms	0.023 ms
Nodo dominante	Hash Join + Seq Scan	Seq Scan on mv_ventas_dia_categoria

```
-----
QUERY PLAN
-----
Limit (cost=1.02..1.03 rows=1 width=250) (actual time=0.014..0.014 rows=1.00 loops=1)
 Buffers: shared hit=1
-> Sort (cost=1.02..1.03 rows=1 width=250) (actual time=0.013..0.013 rows=1.00 loops=1)
   Sort Key: total_vendido DESC
   Sort Method: quicksort Memory: 25kB
   Buffers: shared hit=1
-> Seq Scan on mv_ventas_dia_categoria (cost=0.00..1.01 rows=1 width=250) (actual time=0.008..0.009 rows=1.00 loops=1)
   Filter: (dia = CURRENT_DATE)
   Rows Removed by Filter: 1
   Buffers: shared hit=1
Planning:
 Buffers: shared hit=7
Planning Time: 0.783 ms
Execution Time: 0.023 ms
(14 rows)
```

e) Auditoría

Script EXCEPT ambos sentidos en el .sql. Resultado sobre base migrada: 0 filas.

```
food_store=# (SELECT dia, categoria_id, categoria, total_vendido FROM mv_ventas_dia_categoria
food_store=# EXCEPT
food_store=# SELECT (ped.fecha::date), c.id, c.nombre, SUM(dp.cantidad*dp.precio_unitario)
food_store=# FROM detalle_pedido dp JOIN producto pr ON pr.id=dp.producto_id
food_store=# JOIN categoria c ON c.id=pr.categoria_id JOIN pedido ped ON ped.id=dp.pedido_id
food_store=# WHERE dp.eliminado=FALSE AND ped.eliminado=FALSE
food_store=# GROUP BY (ped.fecha::date), c.id, c.nombre)
food_store=# UNION ALL
food_store=# (SELECT (ped.fecha::date), c.id, c.nombre, SUM(dp.cantidad*dp.precio_unitario)
food_store=# FROM detalle_pedido dp JOIN producto pr ON pr.id=dp.producto_id
food_store=# JOIN categoria c ON c.id=pr.categoria_id JOIN pedido ped ON ped.id=dp.pedido_id
food_store=# WHERE dp.eliminado=FALSE AND ped.eliminado=FALSE
food_store=# GROUP BY (ped.fecha::date), c.id, c.nombre
food_store=# EXCEPT
food_store=# SELECT dia, categoria_id, categoria, total_vendido FROM mv_ventas_dia_categoria);
 dia | categoria_id | categoria | total_vendido
-----+-----+-----+-----
(0 rows)
```